

Robust Spuriousness-Aware Group Discovery for Worst-Group Generalization

Yawen Zhao, Nan Ye, Miao Xu

University of Queensland, Brisbane, Australia.

*Corresponding author(s). E-mail(s): nan.ye@uq.edu.au;
miao.xu@uq.edu.au;

Contributing authors: yawen.zhao@uq.edu.au;

Abstract

Deep neural networks are prone to overfit to spurious correlations in the training data, causing their accuracy to drop sharply on groups of examples where these correlations break. Existing methods often rely on manual group labels to achieve strong worst-group generalization, but such labels can be expensive to obtain, and approaches not using such labels generally perform less well. We propose a novel approach, SIEVE, that achieves strong worst-group generalization without manual group labels, offering the best of both paradigms. Our method first performs robust spuriousness-aware group discovery for a subset of the training examples to create a group-labeled validation set, then uses this validation set to perform model selection. The key to our spuriousness-aware group discovery is a novel observation on the distinct loss dynamics of near-boundary spurious and non-spurious examples. Our approach can be instantiated by combining it with existing methods that perform model selection using group labels in the second step, making these methods applicable without manual group labels. Across six vision and language benchmarks, SIEVE achieves competitive performance with existing methods that rely on manual group labels, and outperforms those that do not.

Keywords: spurious correlation, group robustness, model selection, latent groups

1 Introduction

Deep neural networks often overfit to one or more spurious correlations between non-causal features and labels, leading to severely degraded performance on latent groups

where such correlations break (Shah et al. 2020). For example, an image-based classifier for waterbirds and landbirds may learn to rely on background (water vs. land) instead of the bird itself, spuriously associating land background to landbirds and water background to waterbirds. The classifier thus fails on groups of examples which do not align with the spurious correlations, like “waterbird-on-land” (Sagawa et al. 2020).

To improve worst-group generalization, existing works often use manual group labels defined based on identified spurious correlations for training (Sagawa et al. 2020; Arjovsky et al. 2019; Goel et al. 2021) or model selection (Liu et al. 2021; Kirichenko et al. 2023; Qiu et al. 2023; Noohdani et al. 2024). However, manual group labels only target pre-specified spurious correlations, and they are often expensive to obtain, particularly when domain expertise is required or when multiple spurious correlations are involved. Some works have resorted to discover spurious features (Singla and Feizi 2021; Taghanaki et al. 2022; Wu et al. 2023; Zheng et al. 2024b,a) or latent groups (Sohoni et al. 2020; Creager et al. 2021; Roburin et al. 2022; Yang et al. 2024; Ghaznavi et al. 2025; Zhu et al. 2023), in a self-supervised manner, or with additional knowledge or limited human supervision, and they integrate such discovery mechanisms into the learning process. However, these methods often have weaker worst-group generalization performance than methods using manual group labels.

Motivated by the limitations of the above two paradigms, we ask whether it is possible to design a method that achieves the best of both paradigms, namely, strong worst-group generalization performance and not requiring manual group labels, and we provide an affirmative answer. Our method, **SIEVE** (*Spuriousness-aware Iterative Validation Example selection*), lies at the interface of methods that perform latent group discovery and methods that perform model selection using group labels: it first performs robust spuriousness-aware group discovery for a subset of the training examples to create a group-labeled validation set, then uses this validation set to perform model selection. Our latent group discovery method is designed to provide more robust spuriousness-aware groups: While latent group discovery methods often assign group labels to all training examples, which often results in highly noisy group labels, we focus only on *near-boundary* examples (i.e., examples which are close to other-class examples in the feature space), and leverage a novel observation on the distinct loss dynamics between *spurious* (aligned with a spurious correlation) and *non-spurious* (not aligned with a spurious correlation) examples, as will be described in detail in Section 4. Our approach can be instantiated by combining it with existing methods that perform model selection using group labels in the second step, making these methods applicable without manual group labels. In fact, using our SIEVE validation set instead of a manual validation set leads to competitive worst-group generalization performance in our experiments, highlighting the quality of our SIEVE group labels. This allows SIEVE to inherit the strong performance of methods that use manual group labels, outperforming methods that do not use manual group labels.

We summarize our contributions as follows.

- We identify a loss-dynamics separation between near-boundary spurious and non-spurious examples.

- We propose a novel approach, SIEVE, that leverages the loss-dynamics separation to robustly discover spuriousness-aware group labels for a subset of near-boundary examples, and then learns a model to optimize the worst-group generalization using these spuriousness-aware group labels.
- Empirically, SIEVE achieves competitive performance with existing methods that rely on manual group labels, and outperforms those that do not.

2 Related Work

Model selection with manual group labels. Methods that perform model selection with manual group labels to optimize the worst-group generalization performance use customized training procedures to generate candidate models that are likely to perform robustly in the presence of spurious correlations. JTT (Liu et al. 2021) retrains an ERM (Empirical Risk Minimization) model with its misclassified examples upweighted. AFR (Qiu et al. 2023) retrains only the last layer on a held-out split with a loss-weighted objective that emphasizes poorly predicted examples. DaC (Noohdani et al. 2024) splits each image into causal and context regions, then recomposes causal regions with other-class contexts to synthesize minority-like examples for last-layer retraining. DFR (Kirichenko et al. 2023) retrains the last layer on a group-balanced validation set. SSA (Nam et al. 2022) trains a spurious-attribute estimator from a few group labels and pseudo-labels the rest. CnC (Zhang et al. 2022) aligns same-class examples with dissimilar spurious features through contrastive learning. Although these methods require fewer group labels than those methods that require a group-labeled training set, obtaining the required group labels can still be costly, and these methods may only be effective for handling pre-specified spurious correlations.

Spurious feature or latent group discovery. Existing works that do not use manual group labels mainly perform spurious feature or latent group discovery, in a self-supervised manner, or with additional knowledge or limited human supervision.

Spurious feature discovery methods generally identify raw or high-level features that may encode spurious information, and then reduce the model’s reliance on them. For example, MaskTune (Taghanaki et al. 2022) masks the input regions that a trained model most relies on, treating them as potentially shortcut-related, and then fine-tunes the model to reduce its dependence on those regions. DISC (Wu et al. 2023) flags spurious concepts using a predefined concept bank and intervenes on the training data. SPUME (Zheng et al. 2024b) and LBC (Zheng et al. 2024a) use a vision-language model to detect candidate attributes that may capture spurious information, and use them to guide training.

Latent group discovery methods typically partition or select examples into subsets, and then uses them to modify training. GEORGE (Sohoni et al. 2020) clusters representations learned by an ERM model within each class and applies group distributionally robust optimization over the inferred groups. EIIL (Creager et al. 2021) infers environments that maximally violate an invariant-learning objective for a reference model, then trains an invariant learner. GramClust (Roburin et al. 2022) clusters examples by the Gram matrices of their features, which capture texture and style, then applies robust optimization over the inferred groups. SPARE (Yang

et al. 2024) clusters early-training outputs within each class into pseudo-groups and upweights smaller clusters. EVaLS (Ghaznavi et al. 2025) selects high- and low-loss examples for last-layer retraining and uses inferred environments for model selection. LA-SSL (Zhu et al. 2023) also operates at the example level: it treats slow-learning examples as under-represented or shortcut-conflicting examples and oversamples them during self-supervised pretraining.

There are also methods that do not belong to the above two categories. For example, DPE (To et al. 2025) takes a classifier-design approach: it freezes the ERM-trained representation and replaces the linear classifier with a diverse ensemble of prototype classifiers to improve hidden-subpopulation robustness.

3 Problem Setup

We assume a fixed but unknown distribution P on $(\mathbf{x}, y, g)$, where $\mathbf{x} \in \mathcal{X}$ is the input, $y \in \mathcal{Y}$ is the class label, and $g \in \mathcal{G}$ is a latent group. Typically, each latent group is a subset of a class, though the latent groups may overlap. For example, in the Waterbirds dataset, there are four disjoint latent groups which correspond to the different combinations of the bird type and the background type, while in the CC-WILDS (Koh et al. 2021) dataset, there are 16 overlapping groups, where each group is specified by the value of one of the 8 binary attributes and the value of a binary class label.

In standard classification, the goal is to find a classifier f that maximizes the expected accuracy $\Pr_{(\mathbf{x}, y) \sim P} \mathbb{I}[f(\mathbf{x}) = y]$. This can lead to a classifier that performs poorly on certain groups. Our goal is to find a classifier f that maximizes the worst-group expected accuracy

$$R_{\text{wg}}(f) = \min_{g \in \mathcal{G}} \Pr_{(\mathbf{x}, y) \sim P(\mathbf{x}, y|g)} \mathbb{I}[f(\mathbf{x}) = y]. \quad (1)$$

If the group labels are given, then we can simply optimize the empirical worst-group accuracy. We consider the setting of learning a robust classifier from a dataset $D_{\text{tr}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ that contains only the inputs and their class labels, but not the group labels.

4 Near-boundary Loss Dynamics Separation

The key to SIEVE’s spuriousness-aware group discovery is a novel observation that at the early-training stage, there is a sharp contrast in the loss reduction pattern between *near-boundary examples* (i.e., examples which are close to other-class examples in the feature space): examples that align with spurious correlations show rapid loss reductions, while those that do not align with spurious correlations show much slower loss reductions, and in fact, may experience loss increases. Figure 1a illustrates the loss dynamics separation on the Waterbirds dataset (Sagawa et al. 2020), with illustrations on more real benchmarks provided in Section B.1.

The loss dynamics separation happens because spurious correlations can be exploited by the model as an easy shortcut to quickly fit those examples aligning with

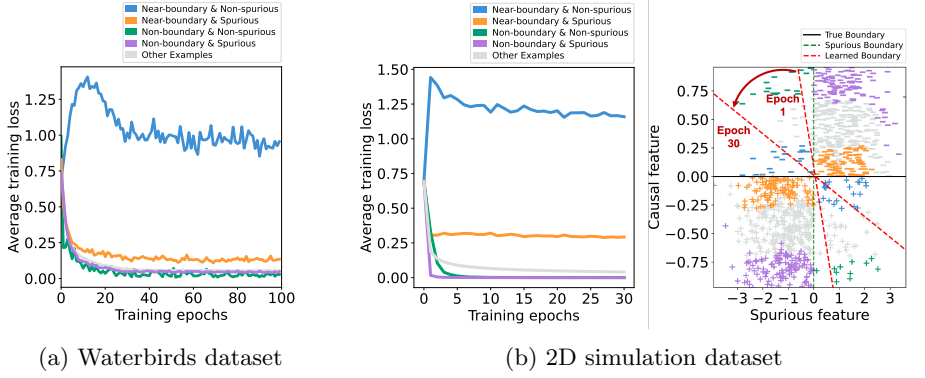


Fig. 1: (a) Comparison of the loss curves for different groups of examples on the Waterbirds dataset, where an example is near-boundary if it is among those 100 with the smallest distance to other-class examples, and non-boundary if it is among those 100 with the largest distance to other-class examples. (b) Left: the same plot as (a) for the 2D simulation dataset. Right: training examples with the true, spurious, and learned boundaries.

them, at the expense of poorer fit for other near-boundary examples, which are much harder to fit because neither spurious correlations nor causal features are particularly discriminative for them. In contrast, for non-boundary examples (i.e., examples which are furthest away from other-class examples), there is little difference between the loss dynamics for the spurious and non-spurious examples. This is because non-boundary examples are far away from other-class examples and easy to fit well compared to boundary examples, as there is no benefit for a learned decision boundary to be close to them, thus they tend to have small losses throughout the learning process.

To better understand the observed loss dynamics separation, we design simulation experiments to answer the following questions:

- How does the learned decision boundary evolve during learning, and how does this lead to the observed loss dynamics separation?
- How does the strength of the spurious correlation affect the strength of the separation?
- Does the loss dynamics separation arise in the presence of multiple possibly interacting spurious features?

We show in Section C that: for (b), the strength of the separation weakens as the spurious correlation weakens; for (c), the separation still arises in the presence of multiple possibly interacting spurious features. Both are expected from our above explanation of why the separation arises, as it only assumes the presence of spurious correlations that can be used as learning shortcuts, and spurious correlations can only offer learning shortcuts when they are sufficiently strong.

Here, we focus on investigating (a) using a 2D simulation benchmark, constructed as follows: each input $\mathbf{x}_i = (x_i^{(\text{spu})}, x_i^{(\text{causal})})$ consists of one causal feature and one

spurious feature. For each example i , we draw a latent scalar $z_i \sim \mathcal{N}(0, 1)$ and set

$$y_i = \text{sign}(z_i) \in \{-1, +1\}, \quad x_i^{(\text{causal})} = \tanh(\alpha z_i), \quad x_i^{(\text{spu})} = a_i |m_i|,$$

where a_i is set to y_i with probability p_{sp} and $-y_i$ otherwise, and $m_i \sim \mathcal{N}(\mu_{\text{sp}}, \sigma_{\text{mag}}^2)$. The feature $x_i^{(\text{causal})}$ alone allows us to determine the class, while the noisy feature $x_i^{(\text{spu})}$ is only correlated with the class. The optimal or causal decision boundary is $x^{(\text{causal})} = 0$, which corresponds to a perfect classifier. The spurious correlation is that the label is $+1$ if $x^{(\text{spu})}$ is positive, and -1 otherwise. This corresponds to the spurious decision boundary $x^{(\text{spu})} = 0$, which only gives correct predictions when $a_i = y_i$.

We generate 10,000 examples with $p_{\text{sp}} = 0.9$, $\alpha = 0.7$, $\mu_{\text{sp}} = 1$ and $\sigma_{\text{mag}}^2 = 1$. We take the 20% examples with smallest distances to other-class examples as the near-boundary set and the 20% with the largest distances to other-class examples as the non-boundary set. A logistic regression model is trained with SGD for 30 epochs, with training details provided in Section C.

Figure 1b (Left) shows that loss dynamics separation between near-boundary spurious examples and near-boundary non-spurious examples is observed on this dataset. Figure 1b (Right) shows the group-colored scatterplot of a dataset and how the learned decision boundary evolves. We can see that at the end of the first epoch, the learned decision boundary has used the spurious correlation as a shortcut to quickly reduce the loss on the near-boundary spurious group, at the expense of increasing the loss on the near-boundary non-spurious group, as shown by the loss curves, and this leads to a decision boundary close to the spurious one $x^{(\text{spu})} = 0$. Such behavior can be explained by the fact that by exploiting the spurious correlation, we can easily fit the large number of near-boundary spurious examples (orange points) well, thus achieving overall loss reduction, even though it can have poor performance on those relatively few near-boundary non-spurious examples (and a few other non-spurious examples). This is a relatively easy way to reduce the overall loss compared to moving to the true decision boundary directly, as that will increase the loss of those many near-boundary spurious examples. However, after the first epoch, the learned decision boundary gradually moves towards the true one. Note that for non-boundary examples, those learned decision boundaries achieve more comparable fit on the spurious ones and the non-spurious ones.

It is interesting to note that the loss dynamics separation arises even with a linear decision boundary, as the fit on the non-boundary examples is strongly influenced by the fit on the near-boundary examples. For a deep neural network, the decision boundary is highly nonlinear, and this could allow the neural network to achieve good fit on non-boundary examples more easily throughout the learning process, without being strongly affected by how the model fits on the near-boundary examples.

5 Learning with Robust Spuriousness-Aware Group Discovery

SIEVE’s pseudocode is shown in Algorithm 1. It first constructs a group-labeled validation set V from the given training set D_{tr} using the LATENTGROUPDISCOVERY and

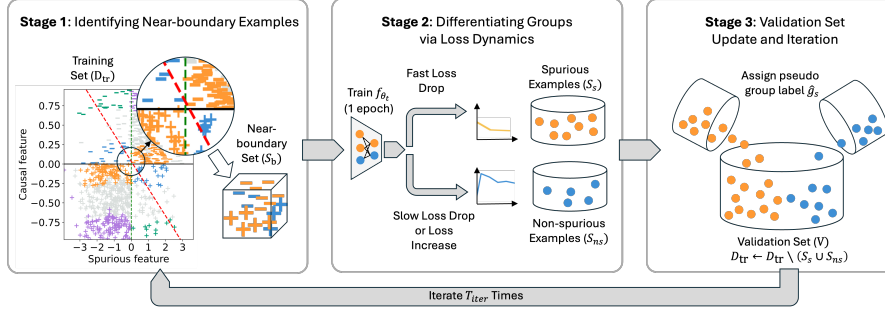


Fig. 2: Overview of SIEVE’s latent group discovery method which iterates through three stages.

removes these examples from D_{tr} , then learns a robust classifier using an algorithm `GENERATEANDSELECTMODEL` by training on a given training set to generate multiple candidate robust models and performing model selection with a group-labeled validation set.

For candidate model generation in the second step, we simply use existing algorithms like ERM or a robust method that performs model selection using group labels (e.g., DaC) in this paper. For ERM, we simply choose the models obtained at the end of each epoch as the candidate models when training a model to minimize the empirical risk on the remaining training data, and we pick the candidate with the highest worst-group accuracy on the validation set. See Algorithm 2 for the complete details. For a robust method that performs model selection using group labels, they already generate candidate models and we simply use their candidate models. In our experiments, we consider JTT (Liu et al. 2021), AFR (Qiu et al. 2023), and DaC (Noohdani et al. 2024).

We explain our latent group discovery method in detail below. An overview of the method is shown in Figure 2. First, it identifies near-boundary examples via feature similarity. Second, it splits these candidates into pseudo spurious and non-spurious groups using early-training loss dynamics. Finally, it assigns latent-group labels and adds the selected examples to the validation set. This process is repeated iteratively. Below, we detail the key design choices and the motivation for the first two steps.

Identifying near-boundary examples. We use a feature extractor f_{θ}^{feat} pretrained on a large-scale dataset (e.g., ImageNet-pretrained ResNet-50) to map inputs to a representation space. Typically, such a pretrained representation is rich and contains both causal and spurious features (e.g., background, texture) for various tasks. For each example $(\mathbf{x}_i, y_i)$, we define its near-boundary score as the distance to the nearest example from any other class:

$$d_i = \min_{j: y_j \neq y_i} \text{dist}(f_{\theta}^{\text{feat}}(\mathbf{x}_i), f_{\theta}^{\text{feat}}(\mathbf{x}_j)). \quad (2)$$

A small d_i means that $\mathbf{x}_i$ is close to another class in the representation space, making it a candidate near-boundary example. In each iteration, we select the N_b examples

Algorithm 1 SIEVE

Input: Training dataset D_{tr} , feature extractor f_{θ}^{feat} , number of iterations T_{iter}

Output: A robust classifier f

```
1:  $D_{\text{tr}}, V \leftarrow \text{LATENTGROUPDISCOVERY}(D_{\text{tr}}, f_{\theta}^{\text{feat}}, T_{\text{iter}})$ 
2:  $f \leftarrow \text{GENERATEANDSELECTMODEL}(D_{\text{tr}}, V)$ 

3: function LATENTGROUPDISCOVERY( $D_{\text{tr}}, f_{\theta}^{\text{feat}}, T_{\text{iter}}$ )
4:   Initialize  $V \leftarrow \emptyset$ 
5:   Extract feature representations:  $\mathbf{Z} \leftarrow f_{\theta}^{\text{feat}}(\mathbf{X})$ 
6:   Compute distance:  $d_i \leftarrow \min_{y_j \neq y_i} \text{dist}(\mathbf{z}_i, \mathbf{z}_j)$ 
7:   for  $t = 1$  to  $T_{\text{iter}}$  do
      // Stage 1: identify near-boundary examples
8:     Compute near-boundary examples  $S_b$  using Equation (3)
      // Stage 2: differentiate groups via loss dynamics
9:     Initialize model  $f_{\theta_t}$  and compute its training loss  $L^{(0)}$  on  $D_{\text{tr}}$ 
10:    Train  $f_{\theta_t}$  on  $D_{\text{tr}}$  for one epoch, then recompute its training loss  $L^{(1)}$  on  $D_{\text{tr}}$ 
11:    Compute loss change:  $\Delta L \leftarrow L^{(1)} - L^{(0)}$ 
12:    Compute spurious examples  $S_s$  and non-spurious examples  $S_{\text{ns}}$  using
      Equations (5) and (6)
      // Stage 3: validation set update
13:    for all  $(\mathbf{x}_i, y_i) \in S_s \cup S_{\text{ns}}$  do
14:      Assign  $\hat{s}_i \leftarrow 0$  if  $i \in S_s$ , else  $\hat{s}_i \leftarrow 1$ 
15:      Assign  $\hat{g}_i$  based on  $y_i$  and  $\hat{s}_i$ 
16:    end for
17:    Update  $V$  to  $V \cup \{(\mathbf{x}_i, y_i, \hat{g}_i)\}_{i \in S_s \cup S_{\text{ns}}}$ , and  $D_{\text{tr}}$  to  $D_{\text{tr}} \setminus (S_s \cup S_{\text{ns}})$ 
18:  end for
19:  return  $D_{\text{tr}}, V$ 
20: end function
```

with the smallest d_i to form the candidate pool S_b :

$$S_b = \{(\mathbf{x}_i, y_i) \mid i \in \text{argmin}_{N_b} \{d_i \mid i \in D_{\text{tr}}\}\}. \quad (3)$$

We define non-boundary examples as the N_{nb} examples with the largest d_i for analysis:

$$S_{\text{nb}} = \{(\mathbf{x}_i, y_i) \mid i \in \text{argmax}_{N_{\text{nb}}} \{d_i \mid i \in D_{\text{tr}}\}\}. \quad (4)$$

Computing the nearest neighbors for all n training examples takes $O(n^2)$ time, but this needs to be performed once only, and this can be done quickly on the datasets used in our experiments. For very large datasets, we note that libraries such as FAISS (Johnson et al. 2021) provide highly efficient implementations of accurate approximate methods that can scale up to millions of vectors, making the nearest neighbors computation overhead of our algorithm small as compared to the training time required. Section B.5 reports the wall-clock cost of SIEVE selection on our four vision benchmarks alongside ERM training time.

Differentiating groups via loss dynamics. To distinguish *near-boundary spurious* from *near-boundary non-spurious* examples within S_b , in iteration t , we first initialize a new model f_{θ_t} and train it for a single epoch on the current training set D_{tr} . We compute the per-sample loss change as $\Delta L_i = L_i^{(1)} - L_i^{(0)}$. Because early training tends to prioritize spurious features, examples aligned with this spurious signal will experience a rapid decrease in loss (a large negative ΔL_i), whereas examples misaligned with it will show a slower decrease or even an increase in loss (a positive ΔL_i). Based on this, we select the top K samples with the most negative ΔL from S_b as the spurious set (S_s):

$$S_s = \{(\mathbf{x}_i, y_i) \mid i \in \operatorname{argmin}_K \{\Delta L_i \mid i \in S_b\}\}, \quad (5)$$

and the top K samples with the most positive ΔL as the non-spurious set (S_{ns}):

$$S_{ns} = \{(\mathbf{x}_i, y_i) \mid i \in \operatorname{argmax}_K \{\Delta L_i \mid i \in S_b\}\}. \quad (6)$$

In our experiments, we parameterize K by a quantile threshold τ : $K = \lceil (1 - \tau) |S_b| \rceil$, the top $(1 - \tau)$ fraction of S_b on each side. We use $\tau = 0.8$ (top 20%); Section 6.4 confirms SIEVE performs robustly with different τ values. These selected samples are assigned latent-group labels $\hat{g}_i$ (derived from their class y_i and identified spuriousness) and moved from the training set to the newly constructed validation set V . We repeat this over T_{iter} rounds rather than selecting all examples at once; because each round’s selected examples are removed from D_{tr} , they are not re-selected in later rounds.

6 Experiments

We perform extensive experiments to evaluate SIEVE’s performance in this section. Section 6.1 compares ERM, JTT, AFR, and DaC, which use manual group labels for model selection, and their SIEVE variants. Section 6.2 compares DaC-SIEVE with multiple group-label-free baselines, namely, GEORGE, EIL, EVaLS, SPARE, and DPE. We also assess SIEVE’s validation-set quality in Section 6.3 and hyperparameter sensitivity in Section 6.4.

We evaluate on six benchmarks (Table 1) that exhibit worst-group performance drops under standard ERM training. Full dataset details are in Section A.2.1.

CivilComments-WILDS (CC-WILDS) has 16 overlapping groups, since a comment may mention several of the eight identities at once. We follow the dataset setups of Sagawa et al. (2020) for Waterbirds, CelebA, and MultiNLI, Noohdani et al. (2024) for Dominoes and MetaShift, and the standard WILDS 16-group setup (Koh et al. 2021) for CC-WILDS.

6.1 Comparison with baselines using manual group labels

We compare SIEVE with four baselines, namely, ERM, JTT (Liu et al. 2021), AFR (Qiu et al. 2023), and DaC (Noohdani et al. 2024). For the baselines, we train a robust model using a manual group-labeled validation set for performing model selection. The candidate models for ERM are the models at the end of each epoch during

Table 1: Benchmarks used in our experiments, with target labels, spurious features, and number of groups $|\mathcal{G}|$. The first four are image tasks and the last two are text tasks.

Dataset	Input	Target	Spurious feature	$ \mathcal{G} $
Waterbirds (Sagawa et al. 2020)	Image	bird type	background type	4
MetaShift (Liang and Zou 2022)	Image	animal category	background type	4
Dominoes (Pagliardini et al. 2023)	Image	vehicle type	MNIST digit	4
CelebA (Liu et al. 2015)	Image	hair color	gender	4
MultiNLI (Williams et al. 2018)	Text	logical relationship	presence of negation	6
CC-WILDS (Koh et al. 2021)	Text	toxicity	demographic identity	16

Table 2: Comparison of validation protocols across vision and text benchmarks. Rows without a superscript use group-labeled validation; † uses average validation accuracy without group labels; “-SIEVE” uses SIEVE’s latent-group labels. Results are mean (std) over 3 seeds. Bold marks the best worst-group accuracy per dataset; *N/A* marks combinations outside scope, since DaC is image-only.

Method	Waterbirds		MetaShift		Dominoes		CelebA		MultiNLI		CC-WILDS	
	Worst	Avg	Worst	Avg	Worst	Avg	Worst	Avg	Worst	Avg	Worst	Avg
ERM	77.83 (1.60)	86.18 (0.56)	66.18 (3.67)	75.53 (1.14)	77.31 (3.76)	88.40 (1.13)	72.59 (4.63)	92.41 (0.21)	68.15 (1.71)	81.30 (0.18)	57.41 (1.48)	92.15 (0.28)
ERM †	71.81 (2.86)	90.83 (0.27)	64.71 (1.86)	75.29 (0.13)	76.16 (2.61)	89.29 (0.44)	65.37 (5.16)	94.44 (0.44)	66.87 (4.51)	83.15 (0.17)	44.61 (3.73)	92.87 (0.04)
ERM-SIEVE	79.84 (2.07)	93.74 (0.46)	65.73 (2.39)	76.26 (1.67)	84.69 (2.92)	91.73 (0.93)	72.59 (1.79)	92.91 (0.14)	68.44 (2.52)	82.04 (0.21)	57.53 (2.01)	91.86 (0.82)
JTT	87.88 (0.26)	90.12 (0.64)	67.36 (4.71)	75.88 (1.69)	78.16 (2.00)	88.68 (0.85)	80.13 (1.65)	89.60 (3.32)	72.50 (1.85)	79.10 (0.30)	67.98 (1.67)	90.00 (1.23)
JTT †	75.31 (1.59)	92.33 (0.35)	65.00 (1.34)	74.98 (0.33)	77.28 (2.35)	89.23 (0.46)	73.00 (3.40)	91.39 (1.69)	64.75 (0.41)	80.52 (0.10)	66.02 (2.29)	90.06 (1.32)
JTT-SIEVE	78.52 (1.51)	93.52 (0.32)	71.62 (3.43)	77.66 (1.13)	84.76 (1.85)	91.92 (0.75)	79.07 (1.72)	86.93 (0.35)	72.16 (1.34)	79.28 (0.06)	68.54 (2.86)	90.00 (1.23)
AFR	88.03 (5.08)	92.47 (1.76)	71.67 (1.00)	72.57 (2.83)	81.00 (3.32)	90.40 (2.42)	82.97 (2.63)	95.20 (0.10)	73.90 (0.46)	81.67 (0.15)	64.10 (2.54)	91.39 (0.32)
AFR †	87.63 (3.93)	92.04 (1.20)	53.33 (12.19)	68.26 (3.28)	72.59 (4.82)	87.19 (0.82)	53.70 (1.79)	94.65 (0.10)	67.36 (2.24)	82.95 (0.15)	58.47 (0.95)	91.97 (0.13)
AFR-SIEVE	88.10 (1.25)	93.17 (0.87)	71.57 (0.61)	72.60 (2.60)	83.73 (3.35)	91.23 (0.84)	82.97 (0.86)	95.20 (0.10)	73.21 (1.07)	81.34 (0.49)	63.93 (2.08)	91.45 (0.24)
DaC	92.30 (0.40)	95.30 (0.40)	78.30 (1.60)	79.30 (0.10)	89.20 (0.10)	92.20 (0.30)	79.08 (1.05)	88.73 (3.56)	<i>N/A</i>			<i>N/A</i>
DaC †	81.83 (1.21)	94.30 (0.54)	73.34 (2.50)	77.10 (0.91)	87.56 (0.29)	92.37 (0.69)	52.78 (3.38)	94.53 (0.04)	<i>N/A</i>			<i>N/A</i>
DaC-SIEVE	91.71 (1.95)	94.06 (1.29)	81.37 (0.60)	82.87 (0.24)	89.59 (0.69)	92.11 (0.56)	78.61 (0.40)	88.90 (0.09)	<i>N/A</i>			<i>N/A</i>

training, while the candidate models for the other methods are the same as in the original methods. Method-specific hyperparameters and implementation details are in Section A.2.2. For these baselines, we also train a standard model obtained by using the classification accuracy on the validation set, instead of the worst-group accuracy on the validation set. For SIEVE, we instantiate it by using each of the baselines for candidate model generation.

Table 2 reports worst-group and average accuracy under three validation protocols: group-labeled validation, average-validation selection without group labels (†), and SIEVE latent-group validation. Dagger-marked rows keep training fixed but select by average validation accuracy; their lower worst-group accuracy exposes the selection gap SIEVE targets. Across six benchmarks, SIEVE-enhanced variants generally preserve the worst-group accuracy of their group-labeled-validation counterparts, with the clearest gains on Dominoes and MetaShift. On Waterbirds, JTT-SIEVE has lower worst-group accuracy but higher average accuracy than JTT. This trade-off likely arises because JTT retrain all model parameters under validation-guided hyperparameter choices, making it more sensitive to latent-group label noise; AFR and DaC only update the last layer and are less sensitive. We provide a more detailed discussion in Section B.4.

Table 3: DaC-SIEVE versus group-label-free baselines on four vision benchmarks. Results are test worst-group and average accuracy, mean (std) over 3 seeds. Baseline rows are our reproductions and may differ from original papers (details in Section A.2.2). Bold marks the best worst-group accuracy per dataset.

Method	Waterbirds		MetaShift		Dominoes		CelebA	
	Worst	Average	Worst	Average	Worst	Average	Worst	Average
DaC-SIEVE	91.71 (1.95)	94.06 (1.29)	81.37 (0.60)	82.87 (0.24)	89.59 (0.69)	92.11 (0.56)	78.61 (0.40)	88.90 (0.09)
GEORGE	84.11 (1.77)	90.42 (0.76)	52.37 (5.96)	64.93 (1.65)	42.11 (2.83)	71.12 (0.66)	55.19 (1.16)	95.21 (0.10)
EIIL	77.83 (2.13)	92.93 (0.08)	72.00 (8.31)	76.59 (3.44)	44.56 (7.05)	72.72 (1.80)	76.48 (3.90)	89.57 (0.80)
EVaLS	88.42 (0.18)	93.63 (0.38)	67.44 (14.96)	77.61 (6.24)	59.86 (1.33)	73.25 (1.31)	69.82 (6.70)	92.87 (1.21)
SPARE	78.87 (1.36)	91.27 (0.27)	57.68 (0.93)	73.33 (0.45)	56.33 (0.54)	76.85 (0.77)	73.89 (2.78)	92.27 (0.29)
DPE	91.53 (1.42)	96.33 (0.15)	81.50 (0.00)	92.60 (0.30)	87.10 (2.44)	92.17 (1.29)	67.93 (2.54)	94.10 (0.00)

Real inputs can contain multiple spurious correlations, even when a benchmark highlights only one. CC-WILDS makes this explicit: one comment can mention several demographic identities, creating multiple identity-based spurious correlations. On CC-WILDS, SIEVE-based selection stays comparable to group-labeled validation selection and well above average-validation selection. These results suggest that SIEVE’s loss-dynamics signal still separates latent groups under co-occurring spurious correlations, without identifying any specific spurious correlation. A controlled 4D study (Section C.4) further shows that the near-boundary loss separation persists with multiple spurious features.

6.2 Comparison with group-label-free baselines

We further compare SIEVE against five group-label-free baselines introduced in Section 2: GEORGE (Sohoni et al. 2020), EIIL (Creager et al. 2021), EVaLS (Ghaznavi et al. 2025), SPARE (Yang et al. 2024), and DPE (To et al. 2025). These baselines cover a representative range of specialized group-label-free approaches: clustering-based group discovery (GEORGE, SPARE), environment inference (EIIL), last-layer retraining (EVaLS), and a prototype-based head (DPE). We use DaC-SIEVE as the comparison anchor. Table 3 reports worst-group and average accuracy on four vision benchmarks. DaC-SIEVE is competitive with or stronger than every group-label-free baseline we test. Among these baselines, DPE is the strongest competitor: DaC-SIEVE matches it within one standard deviation on Waterbirds and MetaShift, and outperforms it on Dominoes and CelebA. These results show that SIEVE-enhanced methods match or outperform strong group-label-free baselines in worst-group accuracy while preserving their training pipelines.

6.3 Quality of SIEVE validation set

We evaluate the quality of SIEVE’s validation set by three measures: (i) SIEVE’s match count with the oracle-selected model (the model selected by worst-group accuracy under the true group labels) and the test-WGA gap between the two (the oracle gap), (ii) the Spearman rank correlation between SIEVE’s latent-group WGA and the oracle WGA across training models, and (iii) the downstream test WGA when SIEVE’s validation set is replaced by alternative latent group proxies.

We compare against two latent group discovery methods: GEORGE (Sohoni et al. 2020) and GramClust (Roburin et al. 2022). GEORGE and GramClust label the same

Table 4: Quality of SIEVE validation sets. Columns report SIEVE’s oracle-selection gap (Match, Gap), Spearman rank correlation with oracle WGA, and downstream WGA when each proxy selects the ERM model. Results are mean (std) over 3 seeds; bold marks the best Spearman correlation and downstream WGA.

Dataset	Oracle gap		Spearman rank correlation			Downstream WGA		
	Match	Gap (pp)	SIEVE	GEORGE	GramClust	SIEVE	GEORGE	GramClust
Waterbirds	2/3	0.79 (1.11)	0.93 (0.03)	0.76 (0.09)	0.85 (0.07)	79.84 (2.07)	77.88 (2.86)	78.71 (2.01)
Dominoes	0/3	0.54 (3.77)	0.93 (0.02)	0.37 (0.12)	0.88 (0.04)	84.69 (2.92)	83.33 (4.66)	84.35 (2.55)
MetaShift	3/3	0.00 (0.00)	0.91 (0.05)	0.71 (0.12)	0.68 (0.11)	65.73 (2.39)	59.36 (12.46)	63.50 (3.69)
CelebA	0/3	0.19 (1.72)	0.54 (0.05)	0.20 (0.14)	0.19 (0.22)	72.59 (1.79)	68.52 (1.79)	68.52 (1.79)

validation examples that SIEVE selects, so that the comparison isolates how examples are partitioned into latent groups rather than the choice of examples. We adapt each method’s group-discovery to label the validation set for model selection: the clustering is fit on the training examples, and each validation example is then assigned to one of the learned clusters. More specifically, for GEORGE, we score each example by its margin to the ERM decision boundary, fit a per-class GMM on the training scores, and take the largest cluster as the majority subgroup and the rest as the minority subgroup, then label each validation example with that GMM. Combining class labels with the subgroup labels yields four latent groups. For GramClust, we cluster all training images by their texture/style (KMeans on projected Gram-matrix features) into two clusters, then assign each validation example to a cluster and cross it with its class. This produces four latent groups, corresponding to the two classes and two clusters.

Oracle gap. In the left columns of Table 4, the test-WGA gap is below 1 percentage point on every dataset, even where SIEVE’s choice differs from the oracle’s.

Spearman rank correlation. In the middle columns, SIEVE remains positive on all datasets and has the strongest rank correlation in each row, while GEORGE and GramClust drop to low values on CelebA.

Downstream WGA. In the right columns, replacing SIEVE’s validation set with either GEORGE’s or GramClust’s groups, with all else fixed, lowers worst-group accuracy on every dataset, most sharply on MetaShift and CelebA.

Together, these results show that SIEVE’s binary latent grouping yields a validation set whose ranking and selection behavior closely tracks the oracle, driving downstream worst-group accuracy. Imperfect per-sample latent-group labels do not prevent SIEVE from selecting near-oracle models, because model selection needs a reliable ordering of candidates, not exact recovery of every subgroup label.

6.4 Sensitivity analysis for SIEVE

To evaluate the robustness of SIEVE, we examine its sensitivity to both the model architecture and the choice of hyperparameters. SIEVE remains highly effective across different configurations (see Section B.2 for full experimental details).

Robustness to Architecture. We replace the ResNet-50 in SIEVE with VGG16 while keeping the downstream model and training protocol fixed. Results in Table 5

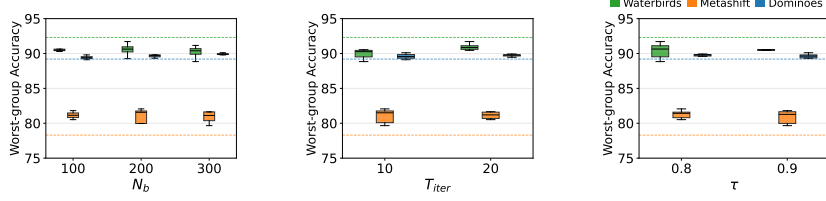


Fig. 3: Sensitivity of DaC-SIEVE to **Left:** N_b , **Middle:** T_{iter} , and **Right:** τ . Boxplots show worst-group accuracy across datasets from the full grid search (Section B.2), excluding $T_{\text{iter}}=5$ due to instability. Dashed lines indicate DaC baselines.

Table 5: Comparison of different feature extractors for SIEVE. While the downstream model (ERM or DaC) remains the same, we vary the architecture used during SIEVE selection (either ResNet-50 or VGG16). All results are averaged over 3 random seeds.

Method	SIEVE Arch	Waterbirds		MetaShift		Dominoes	
		Worst	Avg	Worst	Avg	Worst	Avg
ERM-SIEVE	ResNet-50	79.84 (2.07)	93.74 (0.46)	65.73 (2.39)	76.26 (1.67)	84.69 (2.92)	91.73 (0.93)
ERM-SIEVE	VGG16	79.23 (3.34)	94.06 (0.48)	57.44 (1.89)	72.61 (0.92)	84.15 (0.47)	92.55 (0.09)
DaC-SIEVE	ResNet-50	91.71 (1.95)	94.06 (1.29)	81.37 (0.60)	82.87 (0.24)	89.59 (0.69)	92.11 (0.56)
DaC-SIEVE	VGG16	89.04 (2.13)	94.78 (0.30)	77.26 (6.42)	81.45 (2.63)	89.66 (1.05)	91.97 (0.55)

show that ERM-SIEVE with VGG16 achieves similar worst-group accuracy to ResNet-50 on Waterbirds and Dominoes. On MetaShift, performance drops slightly but remains competitive. Importantly, we directly reused the hyperparameters from ResNet-50 (e.g., learning rate and weight decay) without tuning for VGG16. This highlights that SIEVE is not tightly coupled to a specific backbone architecture.

Robustness to Hyperparameters. We evaluate stability under varying candidate pool sizes N_b , iterations T_{iter} , and thresholds τ . DaC-SIEVE remains stable across most settings, as summarized in Figure 3. The main exception is $T_{\text{iter}}=5$: A small number of iterations is generally insufficient to identify diverse near-boundary examples, leading to lower performance. For instance, on Waterbirds, increasing T_{iter} from 5 to 20 improves worst-group accuracy from 88.68% to 91.71%. Performance then remains stable for $T_{\text{iter}} \geq 10$. Varying N_b from 100 to 300 or τ between 0.8 and 0.9 has no consistent effect on worst-group accuracy once enough selection iterations are used. Full grid-search results are in Section B.2.

7 Conclusion

We introduce SIEVE, a novel approach for optimizing worst-group generalization that lies at the interface of two common approaches: methods that perform model selection using manual group labels, which achieve strong performance, but manual group labels can be expensive; methods that discover latent groups, which avoid acquiring manual group labels, but generally perform less well. Our method uses a novel observation on the loss dynamics of near-boundary examples to reliably discover the latent groups for a subset of examples, and it achieves strong worst-group generalization without manual group labels in our experiments, offering the best of the two existing approaches.

SIEVE demonstrates that by combining reliable discovery of latent spuriousness-aware groups for a small subset of examples and a strong method for generating

candidate robust models, we can achieve strong worst-group generalization without any manual group labels. This general approach can be extended by improving either component, which we believe would be interesting to work on.

Declarations

Funding. ARC DE230101116.

Competing interests. The authors declare no competing interests.

Ethics approval. Not applicable.

Consent to participate. Not applicable.

Consent for publication. Not applicable.

Data availability. All used datasets are publicly available and cited.

Materials availability. Not applicable.

Code availability. Available at <https://github.com/Yawen-Zhao-820/sieve-suppl>.

Authors’ contributions. All authors contributed to all aspects of this work.

Appendix A Additional details

A.1 SIEVE with ERM-based Model Generation

Algorithm 2 shows the pseudocode for how we can instantiate GENERATEANDSELECT-MODEL in SIEVE when using ERM to generate candidate models. We assume that we are performing T rounds of minimization for a risk based on a given loss function.

Algorithm 2 ERM-GenerateAndSelectModel(D_{tr}, V)

- 1: Initialize a model f_θ with parameters θ , and set A_{best} to 0 and θ^* to θ .
 - 2: **for** $t = 1$ to T **do**
 - 3: Perform one round of ERM to update f_θ^{ERM} using the training set D_{tr} .
 - 4: Compute worst-group accuracy: $A_{\text{wg}} = \min_{\hat{g}} \text{Accuracy}(f_\theta^{\text{ERM}}(\mathbf{X}_{\hat{g}}), \mathbf{Y}_{\hat{g}})$, where $\mathbf{X}_{\hat{g}}$ denote all examples in group g in the validation set V , and $\mathbf{Y}_{\hat{g}}$ their corresponding labels.
 - 5: If $A_{\text{wg}} > A_{\text{best}}$, then update A_{best} to A_{wg} and θ^* to θ .
 - 6: **end for**
 - 7: **return** θ^*
-

A.2 Additional experiment details

A.2.1 Datasets

We summarize the details about the datasets we use in Tables A1 to A6. CivilComments-WILDS uses a 4-group partition for SIEVE training and validation-set construction, defined by toxicity and whether the comment mentions any of the eight demographic identities; evaluation follows the WILDS protocol and reports worst-group accuracy over the 16 overlapping toxicity-by-identity groups.

Target: bird type; Spurious feature: background type.				
Group g :	0	1	2	3
Target $y \in \{0, 1\}$:	0 (landbird)	0 (landbird)	1 (waterbird)	1 (waterbird)
Spurious a :	0 (land)	1 (water)	0 (land)	1 (water)
# Train data:	3,498 (73%)	184 (4%)	56 (1%)	1,057 (22%)
# Val data:	467	466	133	133
# Test data:	2,255	2,255	642	642

Table A1: Waterbirds Dataset.

Target: vehicle type; Spurious feature: digit.				
Group g :	0	1	2	3
Target $y \in \{0, 1\}$:	0 (automobile)	0 (automobile)	1 (truck)	1 (truck)
Spurious a :	0 (digit 0)	1 (digit 1)	0 (digit 0)	1 (digit 1)
# Train data:	3,592 (45%)	398 (5%)	398 (5%)	3,592 (45%)
# Val data:	500	499	499	500
# Test data:	490	490	490	490

Table A2: Dominoes Dataset.

Target: animal category; Spurious feature: background type.				
Group g :	0	1	2	3
Target $y \in \{0, 1\}$:	0 (cat)	0 (cat)	1 (dog)	1 (dog)
Spurious a :	0 (sofa / bed)	1 (shelf)	0 (bench / bike)	1 (shelf)
# Train data:	611	0	512	0
# Val data:	0	37	0	44
# Test data:	0	198	0	262

Table A3: MetaShift Dataset.

Target: hair colour; Spurious feature: gender.				
Group g :	0	1	2	3
Target $y \in \{0, 1\}$:	0 (non-blonde)	0 (non-blonde)	1 (blonde)	1 (blonde)
Spurious a :	0 (female)	1 (male)	0 (female)	1 (male)
# Train data:	71,629 (44%)	66,874 (41%)	22,880 (14%)	1,387 (1%)
# Val data:	8,535	8,276	2,874	182
# Test data:	9,767	7,535	2,480	180

Table A4: CelebA Dataset.

A.2.2 Hyperparameter and Hardware Configurations

Common Training Setup. For all experiments on vision datasets, we train a ResNet-50 model pretrained on ImageNet using SGD with a momentum of 0.9. For MultiNLI and CivilComments-WILDS, we use the HuggingFace implementation of the BERT model, initialized with pretrained weights (Devlin et al. 2019). All reported results are averaged across 3 random seeds.

SIEVE Configuration. For all methods that are combined with SIEVE, we set N_b equal to N_{nb} . Specifically, for Dominoes, Waterbirds and CelebA, $N_b = 200$ and

Target: logical relationship; Spurious feature: presence of negation.						
Group g :	0	1	2	3	4	5
Target $y \in \{0, 1, 2\}$:	0 (con)	0 (con)	1 (ent)	1 (ent)	2 (neu)	2 (neu)
Spurious a :	0 (no neg)	1 (neg)	0 (no neg)	1 (neg)	0 (no neg)	1 (neg)
# Train data:	57,498	11,158	67,376	1,521	66,630	1,992
# Val data:	22,814	4,634	26,949	613	26,655	797
# Test data:	34,597	6,655	40,496	886	39,930	1,148

Table A5: MultiNLI Dataset.

Target: toxicity; Spurious feature: mention of a demographic identity.				
Group g :	0	1	2	3
Target $y \in \{0, 1\}$:	0 (non-toxic)	0 (non-toxic)	1 (toxic)	1 (toxic)
Spurious a :	0 (no identity)	1 (identity)	0 (no identity)	1 (identity)
# Train data:	143,628	94,895	11,940	18,575
# Val data:	24,366	15,759	1,967	3,088
# Test data:	72,373	46,185	6,063	9,161

Table A6: CivilComments-WILDS Dataset.

$T_{\text{iter}} = 20$; for MetaShift, $N_b = 100$ and $T_{\text{iter}} = 10$; for MultiNLI, $N_b = 8,000$ and $T_{\text{iter}} = 5$; for CivilComments-WILDS, $N_b = 5,000$ and $T_{\text{iter}} = 5$. We use the quantile-threshold parameterization of K defined in Section 5 with $\tau = 0.8$ in all experiments.

Method-Specific Settings. For JTT, AFR, and DaC, we use each method’s original paper’s reported best method-specific hyperparameters on the datasets that paper evaluates. For datasets outside the corresponding paper, we set a search range following its tuning procedure, with standard training following our ERM configuration. For JTT-SIEVE, AFR-SIEVE, and DaC-SIEVE, we use the corresponding baseline procedure and range on every dataset. We tune hyperparameters and select the final model on the SIEVE-constructed validation set rather than on group-labeled validation data.

- **ERM and ERM-SIEVE.** The learning rate is set to 0.001 for all vision datasets. Weight decay is set to 0.001 for Dominoes, Waterbirds, and MetaShift, and 0.0001 for CelebA. For Dominoes, we use a batch size of 16, train for 15 epochs, and do not apply any data augmentation. For Waterbirds, we use a batch size of 32 and train for 100 epochs. For MetaShift, we use a batch size of 16 and train for 100 epochs. For CelebA, we use a batch size of 128 and train for 30 epochs.

For MultiNLI, we fine-tune BERT for 10 epochs using the AdamW optimizer with a batch size of 32, no weight decay, and an initial learning rate of 2×10^{-5} . For CivilComments-WILDS, we fine-tune BERT for 5 epochs using AdamW with a batch size of 16, weight decay 0.01, and a learning rate of 1×10^{-5} .

- **JTT and JTT-SIEVE.** We follow the official JTT implementation (Liu et al. 2021) where applicable, and set the JTT-specific identification epoch and upweighting factor as follows:

- Waterbirds: epoch 60, upweighting factor 100
- Dominoes: epoch 40, upweighting factor 50
- MetaShift: epoch 60, upweighting factor 50
- CelebA: epoch 1, upweighting factor 50
- MultiNLI: epoch 2, upweighting factor 6
- CivilComments-WILDS: epoch 2, upweighting factor 6

For Dominoes and MetaShift, we use a learning rate of 0.001 and a weight decay of 0.001. For Waterbirds, we use a learning rate of 0.00001 and a weight decay of 1; for CelebA, a learning rate of 0.00001 and a weight decay of 0.1; for MultiNLI, a learning rate of 0.00001 and a weight decay of 0.1.

For CivilComments-WILDS, the JTT identifier uses SGD with momentum 0.9, a batch size of 24, no weight decay, and a learning rate of 2×10^{-5} ; we then identify the error set and fine-tune the final model for 3 epochs using AdamW with linear learning-rate decay, batch size 16, weight decay 0.01, and learning rate 1×10^{-5} .

For JTT-SIEVE on Waterbirds, we use a learning rate of 0.0001, a weight decay of 0.001, 50 epochs, and an upweighting factor of 50. For CelebA, we use 5 epochs for computational efficiency, while keeping all other settings the same. For MultiNLI, we use epoch 2, upweighting factor 4. For the remaining datasets, JTT-SIEVE uses the same settings as JTT.

- **AFR and AFR-SIEVE.** AFR (Qiu et al. 2023) adopts last-layer retraining with a frozen backbone (Stage 1: ERM training; Stage 2: last-layer retraining with a loss-weighted objective). The validation set is used in Stage 2 to tune γ and λ and for model selection. Here, γ controls the degree of upweighting for poorly predicted examples, and λ is an ℓ_2 regularization coefficient on the last-layer parameters during retraining.

For Waterbirds and CelebA, we adopt the Stage 1 and Stage 2 hyperparameters from the original AFR implementation, while for MultiNLI we follow the hyperparameter search protocol described in the AFR paper:

- Waterbirds: Stage 1: batch size 32, learning rate 0.003, weight decay 0.0001, 50 epochs. Stage 2: $\gamma = 10$, $\lambda = 0.0$, 500 epochs.
- CelebA: Stage 1: batch size 32, learning rate 0.003, weight decay 0.0001, 20 epochs. Stage 2: $\gamma = 1.44$, $\lambda = 0.001$, 1000 epochs.
- MultiNLI: Stage 1: batch size 32, learning rate 0.00002, weight decay 0, 5 epochs. Stage 2: learning rate 0.01, 200 epochs; we sweep γ over 10 values linearly spaced in $[10^2, 10^5]$ and λ over 26 values linearly spaced in $[0, 50]$.

For AFR-SIEVE, we select Waterbirds ($\gamma = 4.5$, $\lambda = 0.0$) and CelebA ($\gamma = 1.0$, $\lambda = 0.0$). For Dominoes and MetaShift, Stage 1 follows DaC (Noohdani et al. 2024) and uses the learning rate, weight decay, batch size, and number of epochs listed for those datasets in the ERM and ERM-SIEVE bullet above. In Stage 2, we set batch size 32 and train for 500 epochs. We grid-search γ over 33 values linearly spaced in $[4, 20]$ and $\lambda \in \{0, 0.1, 0.2, 0.3, 0.4\}$. The selected values are:

- AFR: Dominoes ($\gamma = 4.0$, $\lambda = 0.1$); MetaShift ($\gamma = 6.0$, $\lambda = 0.0$).
- AFR-SIEVE: Dominoes ($\gamma = 4.0$, $\lambda = 0.4$); MetaShift ($\gamma = 6.5$, $\lambda = 0.0$).

For CivilComments-WILDS, AFR and AFR-SIEVE use the AdamW ERM checkpoint above as the Stage 1 model and train the Stage 2 last-layer reweighting model

with full-batch gradient descent, learning rate 3×10^{-3} , gradient clipping at 1, $\lambda = 0$, and $\gamma \in \{0, 0.01, 0.1, 1, 3, 10\}$ for 50 epochs. AFR-SIEVE selects $\gamma = 0$.

- **DaC and DaC-SIEVE.** Following the original DaC paper (Noohdani et al. 2024), the ERM stage of DaC uses the same learning rate, weight decay, batch size, and number of epochs listed for ERM and ERM-SIEVE above.

During the last-layer retraining stage of DaC, we use a learning rate of 0.005 for all datasets. A step learning rate scheduler with a step size of 5 and gamma of 0.5 is used. The batch size is set to 64 for all datasets. No regularization terms are applied during the last-layer retraining.

The causal masking flag is set to **True** for the following datasets and hyperparameter settings:

- Waterbirds: 20 epochs, $\alpha = 10$, $q = 0.6$
- Dominoes: 20 epochs, $\alpha = 6$, $q = 0.8$
- MetaShift: 30 epochs, $\alpha = 6$, $q = 0.5$
- CelebA: 15 epochs, $\alpha = 5$, $q = 0.2$

The flag is set to **False** for all other datasets.

Group-Label-Free Baselines. For the group-label-free vision baselines, we follow each method’s original-paper settings where applicable. For Dominoes and MetaShift, which the original papers do not cover, we use our ERM training configuration unless otherwise specified.

- **GEORGE.** We use within-class GMM clustering followed by GroupDRO. For Waterbirds, learning rate 1×10^{-5} , weight decay 1.0, batch size 128, 300 epochs, and $C = 2$. For CelebA, learning rate 1×10^{-5} , weight decay 0.1, batch size 128, 50 epochs, and $C = 3$.
- **EIIL.** We train the environment-inference objective with Adam, learning rate 0.01, and 20,000 steps using a one-epoch ERM reference model, then retrain with GroupDRO and report the final epoch. Retraining uses learning rate 1×10^{-5} and weight decay 1.0 for Waterbirds, learning rate 1×10^{-5} and weight decay 0.1 for CelebA, learning rate 3×10^{-4} for Dominoes, and learning rate 1×10^{-5} and weight decay 1.0 for MetaShift.
- **EVaLS.** We train the ERM model with learning rate 1×10^{-3} , weight decay 1×10^{-4} , and a step learning-rate scheduler, then reinitialize and retrain the last layer with Adam, batch size 32, learning rate 5×10^{-4} , and weight decay 1×10^{-4} . We sweep k and the ℓ_1 penalty over the EVaLS grid and select by validation worst-environment accuracy on EIIL-inferred environments.
- **SPARE.** For Waterbirds, learning rate 1×10^{-4} , weight decay 0.1, batch size 128, 300 epochs, $T_{\text{id}} = 2$, sampling powers [3, 3], and logit clustering. For CelebA, learning rate 1×10^{-5} , weight decay 1.0, batch size 128, 50 epochs, $T_{\text{id}} = 1$, sampling powers [1, 2], and feature clustering. For Dominoes, learning rate 1×10^{-3} , weight decay 1×10^{-3} , batch size 128, 50 epochs, and $T_{\text{id}} = 2$. For MetaShift, learning rate 1×10^{-4} , weight decay 0.1, batch size 128, 300 epochs, and $T_{\text{id}} = 2$.
- **DPE.** We use the no-subgroup variant with the training-attribute flag disabled, the IsoMax loss, frozen ERM features, covariance regularization 10^5 , 16 prototype

heads, learning rate 1×10^{-3} for about 20 epochs, and the last checkpoint as the ensemble criterion.

Hardware Setup. All experiments were conducted on a server equipped with an NVIDIA RTX A6000 GPU (49GB VRAM), an AMD Ryzen Threadripper PRO 3975WX 32-core CPU (64 threads), and 125GB of RAM.

Appendix B Additional analysis of SIEVE

B.1 Loss dynamics analysis

Figure 1a illustrated our key observation on Waterbirds: among near-boundary examples, spurious ones show a faster loss decrease early in training, while non-spurious ones decrease more slowly or even increase. The same holds across Dominoes, MetaShift, CelebA, and MultiNLI (Figure B1), datasets with diverse inputs and shortcuts.

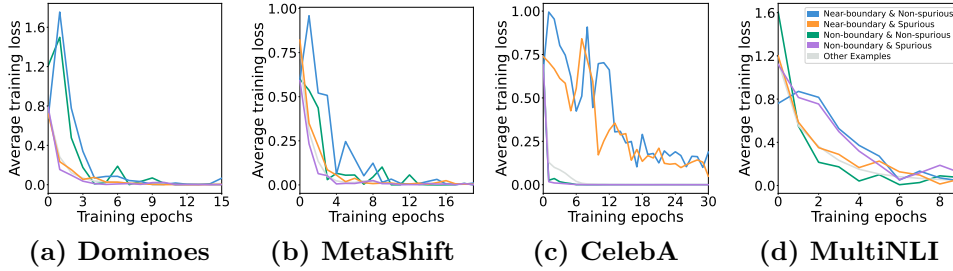


Fig. B1: Loss dynamics on four benchmarks. For each dataset, we plot the average training loss of 100 near-boundary and 100 non-boundary examples, split into spurious and non-spurious groups.

B.2 Complete Sensitivity Analysis

As described in Section 6.4, we conducted comprehensive sensitivity analyses for SIEVE under different hyperparameter settings. Due to space limitations in the main paper, we present the full set of results here in Table B7. These results cover various combinations of $\tau \in \{0.8, 0.9\}$, $N_b \in \{100, 200, 300\}$, and $T_{\text{iter}} \in \{5, 10, 20\}$, evaluated on Waterbirds, Dominoes, and MetaShift.

B.3 Stability of SIEVE with respect to data ordering.

SIEVE uses the one-epoch loss change ΔL only to rank training examples; thus, its practical stability is reflected directly in the stability of the selected validation set. To assess the stability of SIEVE with respect to data ordering, we reuse the three random seeds from our Waterbirds experiments.

For each seed, we compare the resulting sets pairwise. Table B8 reports the Jaccard similarity and percentage overlap between the selected validation sets for each pair of seeds. We observe Jaccard similarities above 0.8 and overlaps around 90% across

Table B7: We report the average worst-group accuracy (Worst) and average accuracy (Average) over 3 random seeds on each dataset under different SIEVE configurations. The **bold** numbers correspond to the results from the main experiments: $N_b = 200$, $T_{\text{iter}} = 20$, $\tau = 0.8$ for Waterbirds and Dominoes, and $N_b = 100$, $T_{\text{iter}} = 10$, $\tau = 0.8$ for MetaShift. These values are copied from Table 2 for consistency.

Method	N_b	T_{iter}	τ	Waterbirds		MetaShift		Dominoes	
				Worst	Average	Worst	Average	Worst	Average
DaC	-	-	-	92.30 (0.40)	95.30 (0.40)	78.30 (1.60)	79.30 (0.10)	89.20 (0.10)	92.20 (0.30)
DaC-SIEVE ($\tau = 0.8$)	100	5	0.8	89.72 (1.48)	95.20 (0.17)	78.32 (3.35)	82.93 (2.24)	87.01 (2.77)	90.53 (0.86)
	200	5	0.8	88.68 (1.96)	94.66 (0.23)	79.86 (2.46)	83.61 (1.98)	89.59 (1.14)	92.24 (0.45)
	300	5	0.8	89.72 (1.53)	94.87 (0.36)	78.65 (3.02)	82.99 (2.00)	86.12 (3.01)	90.53 (0.16)
	100	10	0.8	90.29 (1.71)	94.80 (0.49)	81.37 (0.60)	82.87 (0.24)	89.11 (0.12)	92.64 (0.16)
	200	10	0.8	89.27 (1.02)	93.95 (2.02)	82.06 (0.68)	83.98 (0.75)	89.80 (0.89)	92.35 (0.53)
	300	10	0.8	88.84 (0.78)	95.05 (0.20)	81.63 (0.75)	83.92 (0.64)	89.87 (1.03)	91.50 (0.74)
	100	20	0.8	90.99 (0.97)	93.83 (0.42)	80.52 (1.44)	82.07 (0.98)	89.80 (0.89)	92.35 (0.53)
	200	20	0.8	91.71 (1.95)	94.06 (1.29)	81.48 (0.38)	83.30 (1.98)	89.59 (0.69)	92.11 (0.56)
	300	20	0.8	91.16 (1.13)	93.86 (0.14)	80.61 (2.47)	82.44 (1.92)	89.93 (0.72)	92.47 (0.64)
DaC-SIEVE ($\tau = 0.9$)	100	5	0.9	89.98 (2.03)	94.82 (0.40)	76.24 (6.53)	83.73 (0.85)	82.99 (4.60)	88.93 (2.83)
	200	5	0.9	89.72 (1.53)	94.78 (0.45)	74.12 (2.77)	81.95 (2.68)	84.63 (2.74)	90.22 (0.61)
	300	5	0.9	90.21 (1.67)	94.50 (1.05)	78.26 (2.25)	81.82 (2.48)	84.63 (2.74)	90.22 (0.61)
	100	10	0.9	90.50 (1.64)	94.73 (0.51)	81.81 (0.50)	83.36 (0.67)	89.32 (0.43)	92.99 (0.61)
	200	10	0.9	90.55 (1.56)	94.76 (0.47)	75.42 (5.82)	80.71 (2.77)	89.32 (0.43)	92.99 (0.61)
	300	10	0.9	90.24 (1.67)	94.61 (0.68)	79.66 (3.91)	82.50 (2.14)	90.11 (0.84)	93.87 (1.77)
	100	20	0.9	90.45 (0.99)	94.43 (0.71)	80.92 (1.94)	84.75 (0.39)	89.46 (1.33)	92.35 (0.53)
	200	20	0.9	90.71 (0.78)	94.41 (0.52)	81.66 (0.92)	83.24 (2.21)	89.80 (0.89)	92.35 (0.53)
	300	20	0.9	90.55 (1.10)	94.74 (0.18)	81.63 (0.75)	83.92 (0.64)	89.80 (0.89)	92.35 (0.53)

Table B8: Stability of SIEVE validation sets on Waterbirds across three random seeds.

Seed pair	(1, 2)	(1, 3)	(2, 3)
Jaccard similarity	0.83	0.80	0.84
Overlap (%)	90.49	89.00	91.09

all seed pairs, indicating that ΔL produces a consistent ranking and is not driven by mini-batch ordering randomness.

B.4 Analysis of JTT-SIEVE on Waterbirds

The observed performance drop arises because JTT retraines the entire model guided by the validation set, so imperfections in the latent-group-labeled validation set can affect all model parameters; in contrast, AFR and DaC only update the last layer and are less affected.

In detail, JTT starts from a standard ERM model and then retraines the entire model in a second stage. It uses the validation set to choose (i) the first-stage training length T , (ii) the second-stage upweighting factor λ , and (iii) the early-stopping epoch in the second stage. By contrast, both AFR and DaC are last-layer retraining methods and there is no second stage in which a new full model is trained. Therefore, AFR-SIEVE and DaC-SIEVE achieve worst-group accuracies that are very close to those of AFR and DaC when using the same SIEVE latent-group labels.

More specifically, AFR starts from a standard ERM model and retrains only the last layer with a weighted loss that emphasizes examples where the base model performs poorly, which implicitly upweights the minority group. The validation set is used to tune the reweighting strength and for early stopping. DaC is also a last-layer retraining method. In DaC, the validation set guides counterfactual data augmentation and early stopping: it uses worst-group accuracy to select (i) the fraction of samples per batch used to construct mixed images, (ii) the loss weight on mixed images, and (iii) whether to apply or invert the saliency mask. In both methods, the validation signal only affects a small number of hyperparameters on top of a fixed backbone, rather than driving the training of a new model from scratch. Consequently, AFR-SIEVE and DaC-SIEVE are much less sensitive than JTT-SIEVE when the available non-spurious signal is limited and noisy.

B.5 Running time

To better understand the computational overhead introduced by SIEVE, we report the running time (in minutes) for both the SIEVE selection phase and standard ERM training across four datasets in Table B9. While SIEVE incurs some additional computation, its cost remains modest compared to the full training time. Notably, on datasets like MetaShift and Dominoes, the selection time is under 5 minutes, demonstrating the efficiency and practicality of our method.

Table B9: Average running time (in minutes) over 3 random seeds for SIEVE selection and ERM training on each dataset. Results are reported as mean (standard deviation).

Dataset	SIEVE Selection Time	ERM Training Time
Waterbirds	11.28 (0.04)	82.17 (1.18)
MetaShift	2.22 (0.13)	30.43 (1.30)
Dominoes	4.36 (0.19)	11.91 (0.20)
CelebA	104.13 (0.22)	426.96 (0.13)

B.6 Limitations

While SIEVE is simple and effective, it has several limitations. First, its latent-group labels carry noise. Although the validation set still ranks models reliably (Section 6.3), this noise may affect downstream methods that rely on precise group labels. Second, SIEVE assumes that loss dynamics reflect spurious versus non-spurious behavior. While this holds across the benchmarks commonly used in the literature, it may not generalize to some extreme cases, such as data with high noise or severe class imbalance. Future work may focus on reducing latent-group label noise and identifying signals beyond training loss for settings where loss dynamics do not cleanly separate spurious from non-spurious examples.

Appendix C Simulation studies

Section 4 gives the 2D construction and visualization. Here, we provide the higher-dimensional extensions: a 3D dataset that varies the prevalence and noise of one spurious correlation, and a 4D dataset with two spurious correlations.

C.1 Synthetic dataset construction

The 3D and 4D extensions use the same training protocol and group definitions as the 2D study, but with higher-dimensional features.

3D dataset with a single spurious correlation. To study when the loss dynamics hold, we construct a 3D synthetic dataset with each $(\mathbf{x}_i, y_i) \in \mathbb{R}^3 \times \{-1, +1\}$ generated from a latent variable z_i :

$$z_i \sim \text{Unif}(-1, 1), \quad y_i = \text{sign}(\sin(5z_i) + \exp(z_i)).$$

The feature vector $\mathbf{x}_i = (x_i^{(1)}, x_i^{(2)}, x_i^{(3)})$ has one causal feature and two dimensions that form one spurious correlation:

$$x_i^{(1)} = \tanh(\sin(5z_i) + \exp(z_i)), \quad x_i^{(2)} = b_i \sin(5z_i) + \epsilon_i^{(2)}, \quad x_i^{(3)} = b_i \exp(z_i) + \epsilon_i^{(3)},$$

where $b_i = +1$ with probability p_{sp} and -1 otherwise, and $\epsilon_i^{(2)}, \epsilon_i^{(3)} \sim \mathcal{N}(0, \sigma_{\text{sp}}^2)$ independently. Unlike the 2D setup, these spurious dimensions include additive Gaussian noise. Example i is *spurious* if $\text{sign}(x_i^{(2)} + x_i^{(3)}) = y_i$ and *non-spurious* otherwise.

4D dataset with two spurious correlations. The 4D dataset extends the 3D construction by keeping z_i, y_i , and $(x_i^{(1)}, x_i^{(2)}, x_i^{(3)})$ as above, with p_{sp} and σ_{sp} governing the first spurious correlation $(x_i^{(2)}, x_i^{(3)})$. It adds a second spurious feature

$$x_i^{(4)} = b_i^{(4)}(-2[\sin(5z_i) + \exp(z_i)]) + \epsilon_i^{(4)},$$

where $b_i^{(4)} = +1$ with probability $p_{\text{sp}}^{(4)}$ and -1 otherwise, and $\epsilon_i^{(4)} \sim \mathcal{N}(0, (\sigma^{(4)})^2)$. The full input is $\mathbf{x}_i = (x_i^{(1)}, x_i^{(2)}, x_i^{(3)}, x_i^{(4)}) \in \mathbb{R}^4$. We declare example i *spurious* if either $\text{sign}(x_i^{(2)} + x_i^{(3)})$ or $\text{sign}(-x_i^{(4)})$ predicts y_i . The two spurious correlations partially overlap because they share the same latent variable z_i .

C.2 Training protocol

Model and optimization. For each configuration, we generate 10,000 examples and train logistic regression on inputs $\mathbf{x}_i$ using SGD for 10 epochs (learning rate 5×10^{-1} , batch size 128). We initialize all weights to zero, making the initial predictor uninformative and the early loss dynamics data-driven.

Near-boundary and non-boundary examples. For each synthetic dataset, we compute d_i as in Equation (2). Examples in the bottom 20% by d_i are near-boundary, and those in the top 20% are non-boundary.

Loss dynamics and summary statistic. For each epoch, we track the average training loss for the four groups. Following Section 5, we use the one-epoch loss change $\Delta L_i^{(t_0)}$ with $t_0 = 1$. Let $\mathcal{C}$ denote the near-boundary set. We summarize the near-boundary separation by

$$\Delta_{\text{conf}} := \mathbb{E}[\Delta L_i^{(t_0)} \mid i \in \mathcal{C}, i \text{ non-spurious}] - \mathbb{E}[\Delta L_i^{(t_0)} \mid i \in \mathcal{C}, i \text{ spurious}]. \quad (\text{C1})$$

Larger values indicate a stronger gap between near-boundary non-spurious and spurious examples. We show group-wise loss curves in Figures C2 and C4 and report Δ_{conf} for the 3D experiments in Figure C3.

C.3 Results on the 3D dataset.

Varying the prevalence and noise level of one spurious correlation. We study when the loss dynamics hold or fail in the 3D dataset by varying two construction parameters: the prevalence p_{sp} and noise level σ_{sp} of the spurious correlation. Together they determine how easily the model can exploit the spurious signal. We sweep $p_{\text{sp}} \in \{0.9, 0.8, 0.7, 0.5\}$ and $\sigma_{\text{sp}} \in \{0, 0.5, 1.0, 2.0\}$; Figure C2 shows the resulting loss curves.

When p_{sp} is high and σ_{sp} is small, near-boundary spurious examples drop rapidly in loss, while near-boundary non-spurious examples improve slowly or even increase, matching the 2D pattern. As p_{sp} decreases or σ_{sp} increases, this near-boundary gap shrinks; in low-prevalence, high-noise settings, the two near-boundary groups follow similar trajectories. Figure C3 summarizes this trend with $\Delta_{\text{conf}}(t_0 = 1)$. The separation is strongest when the spurious pattern is both prevalent and clean, and decreases as either prevalence or reliability is reduced.

C.4 Results on the 4D dataset

The trained model fits both spurious features. We first verify that the model relies on both spurious features in the 4D setting. We set $p_{\text{sp}} = p_{\text{sp}}^{(4)} = 0.8$ and $\sigma_{\text{sp}} = \sigma^{(4)} = 0$, making both spurious correlations strong and noiseless, and inspect the learned logistic-regression weights at epochs 1 and 5. The weights on the first spurious feature $(x^{(2)}, x^{(3)})$ are $(0.92, 0.38)$ at epoch 1 and $(0.98, 0.68)$ at epoch 5; the weight on the second spurious feature $x^{(4)}$ is -0.31 at epoch 1 and -0.44 at epoch 5. The signs match each feature’s correlation with the label, and the magnitudes are comparable, confirming that the model exploits both shortcuts rather than only one.

Two spurious correlations in the four-dimensional dataset. We next test whether the loss separation persists when two spurious correlations co-occur. We fix the first correlation $(x_i^{(2)}, x_i^{(3)})$ at $p_{\text{sp}} = 0.8$ and $\sigma_{\text{sp}} = 0$, set $p_{\text{sp}}^{(4)} = 0.8$ for the second, and vary $\sigma^{(4)} \in \{0, 0.5, 1.0, 2.0\}$, as shown in Figure C4.

Across noise levels, the near-boundary separation remains stable: near-boundary spurious examples show a larger early loss drop than near-boundary non-spurious examples, with mild variability at large $\sigma^{(4)}$. Increasing $\sigma^{(4)}$ mainly affects non-boundary examples. When $\sigma^{(4)} = 0$, non-boundary spurious examples decrease faster because the second shortcut is clean; as $\sigma^{(4)}$ increases, their curve moves closer to

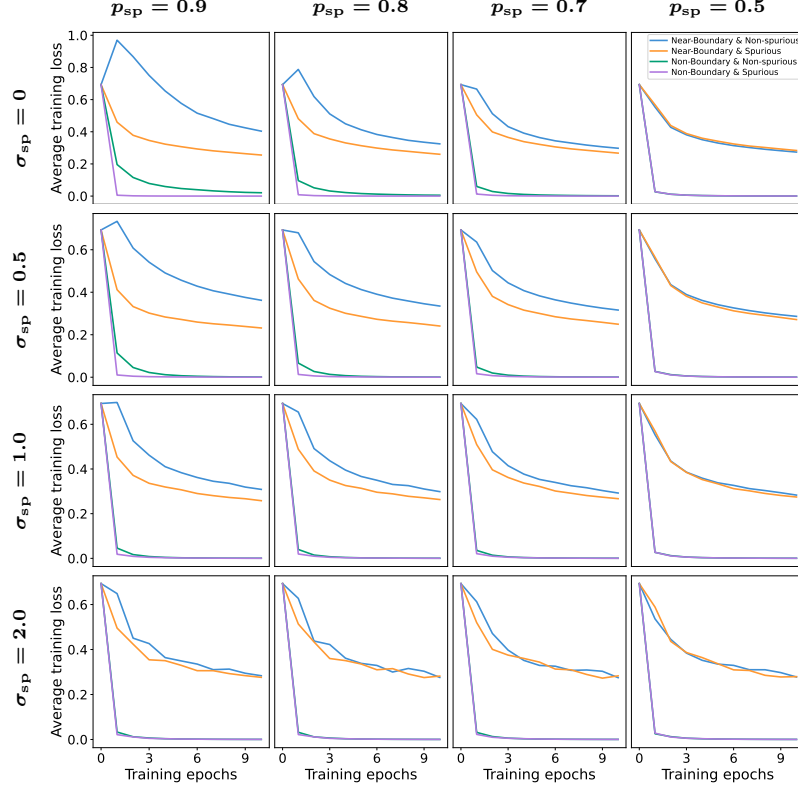


Fig. C2: Average training loss over epochs on the synthetic dataset for different combinations of spurious prevalence p_{sp} (columns) and spurious noise level σ_{sp} (rows). Each panel shows the four groups: Near-boundary & Spurious, Near-boundary & Non-spurious, Non-boundary & Spurious, and Non-boundary & Non-spurious.

the non-boundary non-spurious curve. These results suggest that the near-boundary loss-dynamics separation still persists when multiple spurious correlations co-occur.

References

- Arjovsky, M., Bottou, L., Gulrajani, I., Lopez-Paz, D.: Invariant risk minimization. arXiv (2019)
- Creager, E., Jacobsen, J.-H., Zemel, R.: Environment inference for invariant learning. In: ICML 2021 (2021)
- Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. In: NAACL 2019 (2019)
- Ghaznavi, M., Asadollahzadeh, H., Noohdani, F.H., Tabar, S.V., Hasani, H., Alvanagh, T.A., Rohban, M.H., Baghshah, M.S.: Exploiting what trained models learn for making them robust to spurious correlations without group annotations. In: ICLR 2025 Workshop on Spurious Correlation and Shortcut Learning (2025)

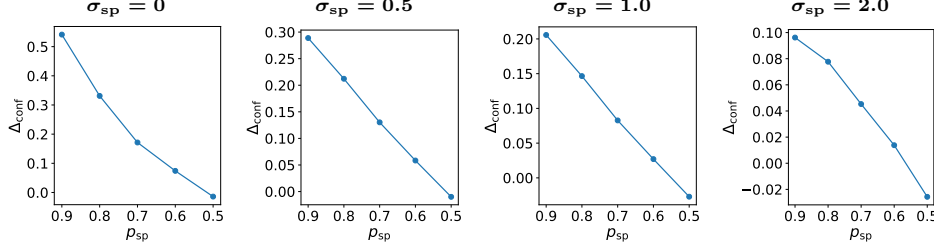


Fig. C3: Separation in early loss dynamics on the synthetic dataset. For each spurious noise level σ_{sp} , we plot the gap $\Delta_{\text{conf}}(t_0 = 1)$ in Equation (C1).

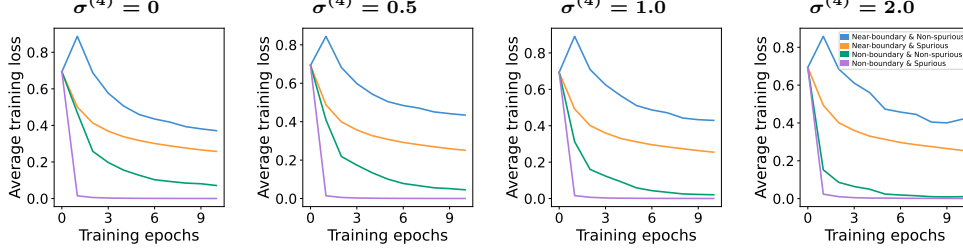


Fig. C4: Four-dimensional construction with two spurious correlations. We fix the first correlation ($x^{(2)}, x^{(3)}$) and vary the noise of the second, $x^{(4)}$. The near-boundary loss separation persists across noise levels.

- Goel, K., Gu, A., Li, Y., Ré, C.: Model patching: Closing the subgroup performance gap with data augmentation. In: ICLR 2021 (2021)
- Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. IEEE Transactions on Big Data **7**(3), 535–547 (2021)
- Kirichenko, P., Izmailov, P., Wilson, A.G.: Last layer re-training is sufficient for robustness to spurious correlations. In: ICLR 2023 (2023)
- Koh, P.W., Sagawa, S., Marklund, H., Xie, S.M., Zhang, M., Balsubramani, A., Hu, W., Yasunaga, M., Phillips, R.L., Gao, I., *et al.*: Wilds: A benchmark of in-the-wild distribution shifts. In: ICML 2021 (2021)
- Liu, E.Z., Haghighi, B., Chen, A.S., Raghunathan, A., Koh, P.W., Sagawa, S., Liang, P., Finn, C.: Just train twice: Improving group robustness without training group information. In: ICML 2021 (2021)
- Liu, Z., Luo, P., Wang, X., Tang, X.: Deep learning face attributes in the wild. In: ICCV 2015 (2015)
- Liang, W., Zou, J.: Metashift: A dataset of datasets for evaluating contextual distribution shifts and training conflicts. In: ICLR 2022 (2022)
- Noohdani, F.H., Hosseini, P., Parast, A.Y., Araghi, H.Y., Baghshah, M.S.: Decompose-and-compose: A compositional approach to mitigating spurious correlation. In: CVPR 2024 (2024)
- Nam, J.H., Kim, J., Lee, J., Shin, J.: Spread spurious attribute: Improving worst-group

- accuracy with spurious attribute estimation. In: ICLR 2022 (2022)
- Pagliardini, M., Jaggi, M., Fleuret, F., Karimireddy, S.P.: Agree to disagree: Diversity through disagreement for better transferability. In: ICLR 2023 (2023)
- Qiu, S., Potapczynski, A., Izmailov, P., Wilson, A.G.: Simple and fast group robustness by automatic feature reweighting. In: ICML 2023 (2023)
- Roburin, S., Corbière, C., Puy, G., Thome, N., Aubry, M., Marlet, R., Pérez, P.: Take one gram of neural features, get enhanced group robustness. arXiv (2022)
- Sohoni, N., Dunnmon, J., Angus, G., Gu, A., Ré, C.: No subclass left behind: Fine-grained robustness in coarse-grained classification problems. In: NeurIPS 2020 (2020)
- Singla, S., Feizi, S.: Salient ImageNet: How to discover spurious features in Deep Learning? arXiv preprint arXiv:2110.04301 (2021)
- Sagawa, S., Koh, P.W., Hashimoto, T.B., Liang, P.: Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization. In: ICLR 2020 (2020)
- Shah, H., Tamuly, K., Raghunathan, A., Jain, P., Netrapalli, P.: The pitfalls of simplicity bias in neural networks. In: NeurIPS 2020 (2020)
- Taghanaki, S.A., Khani, A., Khani, F., Gholami, A., Tran, L., Mahdavi-Amiri, A., Hamarneh, G.: Masktune: Mitigating spurious correlations by forcing to explore. In: NeurIPS 2022 (2022)
- To, N.N.M., Wilson, P.F.R., Nguyen, V., Harmanani, M., Cooper, M., Fooladgar, F., Abolmaesumi, P., Mousavi, P., Krishnan, R.G.: Diverse prototypical ensembles improve robustness to subpopulation shift. In: ICML 2025 (2025)
- Williams, A., Nangia, N., Bowman, S.: A broad-coverage challenge corpus for sentence understanding through inference. In: NAACL 2018 (2018)
- Wu, S., Yükeşgönül, M., Zhang, L., Zou, J.: Discover and cure: Concept-aware mitigation of spurious correlation. In: ICML 2023 (2023)
- Yang, Y., Gan, E., Dziugaite, G.K., Mirzasoleiman, B.: Identifying spurious biases early in training through the lens of simplicity bias. In: AISTATS 2024 (2024)
- Zhu, W., Liu, S., Fernandez-Granda, C., Razavian, N.: Making self-supervised learning robust to spurious correlation via learning-speed aware sampling. arXiv (2023)
- Zhang, M., Sohoni, N.S., Zhang, H.R., Finn, C., Ré, C.: Correct-n-contrast: a contrastive approach for improving robustness to spurious correlations. In: ICML 2022 (2022)
- Zheng, G., Ye, W., Zhang, A.: Learning robust classifiers with self-guided spurious correlation mitigation. In: IJCAI 2024 (2024)
- Zheng, G., Ye, W., Zhang, A.: Spuriousness-aware meta-learning for learning robust classifiers. In: KDD 2024 (2024)